Homework 08

Question 1

(a) The distance to the boundary is $|f(x)|/\sqrt{0.8^2 + 0.6^2} = |f(x)|$. The functional margin is $yf(x)$: values below 1 are inside the class margin, values equal to 1 are on it, and values above 1 are beyond it.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

data = pd.read_csv("margin-geometry-20.csv")
score = 0.8 * data["x1"] + 0.6 * data["x2"]
data["distance"] = score.abs()
data["functional_margin"] = data["y"] * score
on_margin = np.isclose(data["functional_margin"], 1, rtol=0, atol=1e-10)
data["position"] = np.where(
    on_margin, "on",
    np.where(data["functional_margin"] < 1, "inside", "beyond")
)
from IPython.display import Markdown
result_table = data[["observation", "y", "distance", "functional_margin", "position"]].copy()
for column in ["distance", "functional_margin"]:
    result_table[column] = result_table[column].map("{:.2f}".format)
header = "| " + " | ".join(result_table.columns) + " |"
separator = "| " + " | ".join(["---"] * len(result_table.columns)) + " |"
rows = ["| " + " | ".join(map(str, row)) + " |" for row in result_table.itertuples(index=False, name=None)]
display(Markdown("\n".join([header, separator] + rows)))
```

observation	y	distance	functional_margin	position
1	1	1.00	1.00	on
2	1	1.35	1.35	beyond
3	1	1.50	1.50	beyond
4	1	1.70	1.70	beyond
5	1	1.90	1.90	beyond
6	1	2.20	2.20	beyond
7	1	2.40	2.40	beyond
8	1	0.64	0.64	inside
9	1	0.20	0.20	inside
10	1	0.20	-0.20	inside
11	-1	1.00	1.00	on
12	-1	1.30	1.30	beyond
13	-1	1.55	1.55	beyond
14	-1	1.75	1.75	beyond
15	-1	1.95	1.95	beyond
16	-1	2.15	2.15	beyond
17	-1	2.45	2.45	beyond

observation	y	distance	functional_margin	position
18	-1	0.64	0.64	inside
19	-1	0.24	0.24	inside
20	-1	0.20	-0.20	inside

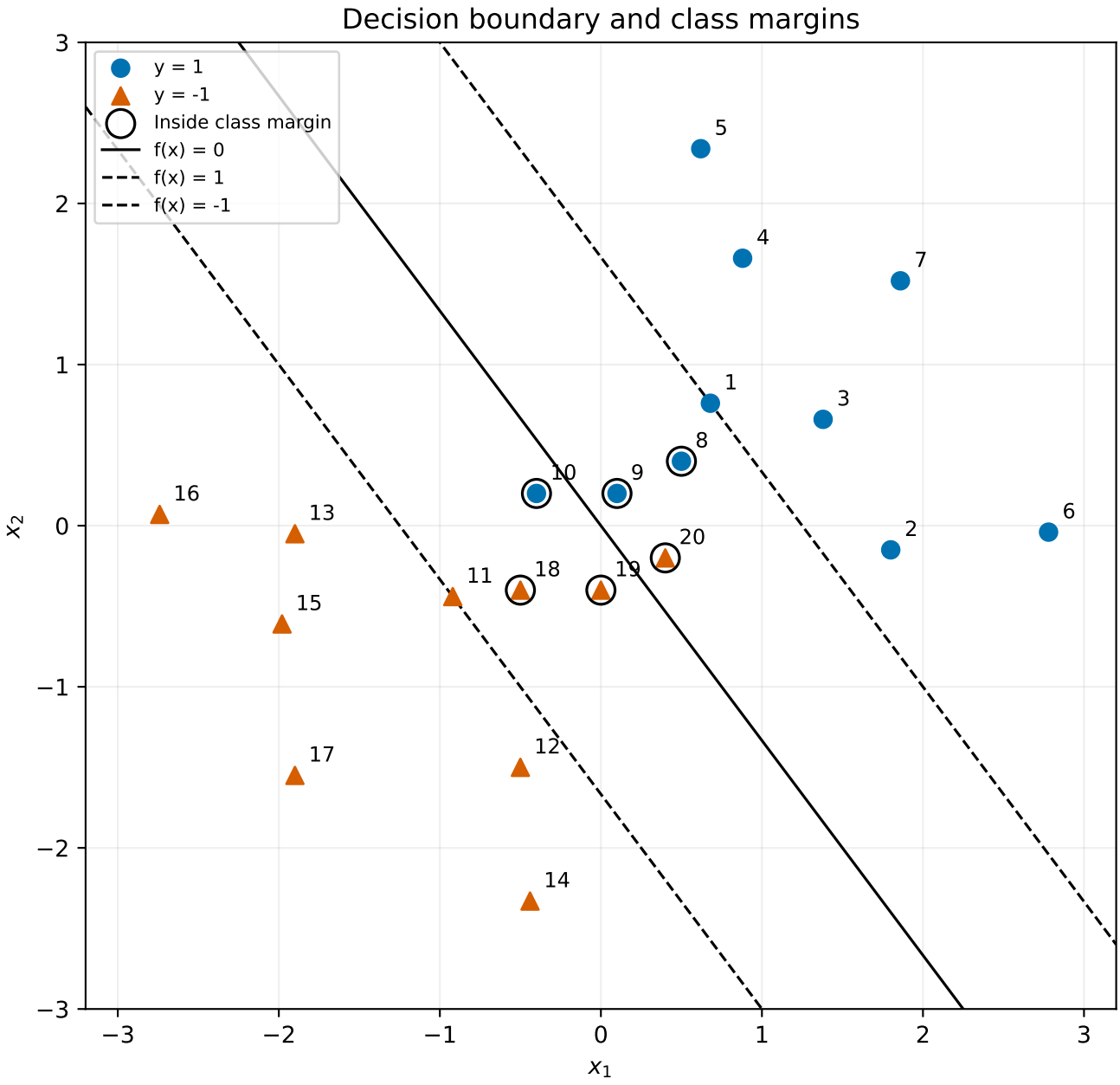
(b) The outlined points are inside their class margin. The labels are the observation numbers from the data.

```
fig, ax = plt.subplots(figsize=(7, 7))
for label, color, marker in [(1, "#0072B2", "o"), (-1, "#D55E00", "^")]:
    group = data[data["y"] == label]
    ax.scatter(group["x1"], group["x2"], color=color, marker=marker,
               s=55, label=f"y = {label}", zorder=3)

inside = data[data["position"] == "inside"]
ax.scatter(inside["x1"], inside["x2"], s=150, facecolors="none",
           edgecolors="black", linewidths=1.2, label="Inside class margin", zorder=4)
for row in data.itertuples():
    ax.annotate(str(row.observation), (row.x1, row.x2),
                xytext=(6, 6), textcoords="offset points", fontsize=9)

x1_grid = np.linspace(-3.2, 3.2, 300)
for level, linestyle in [(0, "-"), (1, "--"), (-1, "--")]:
    ax.plot(x1_grid, (level - 0.8 * x1_grid) / 0.6,
            color="black", linestyle=linestyle, linewidth=1.2,
            label=f"f(x) = {level}")

ax.set(xlim=(-3.2, 3.2), ylim=(-3, 3), xlabel=r"$x_1$", ylabel=r"$x_2$",
       title="Decision boundary and class margins")
ax.set_aspect("equal", adjustable="box")
ax.legend(loc="upper left", fontsize=8)
ax.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```



(c) There are **6 inside**, **2 on**, and **12 beyond** the margin. Observations **10 and 20** are misclassified because their functional margins are negative. Observation 10 has $y = 1$ but $f(x) = -0.20$, while observation 20 has $y = -1$ but $f(x) = 0.20$.

```
print(data["position"].value_counts().reindex(["inside", "on", "beyond"]).to_string())
print("Misclassified observations:",
      data.loc[data["functional_margin"] < 0, "observation"].tolist())
```

```
position
inside     6
on         2
beyond    12
Misclassified observations: [10, 20]
```

Question 2

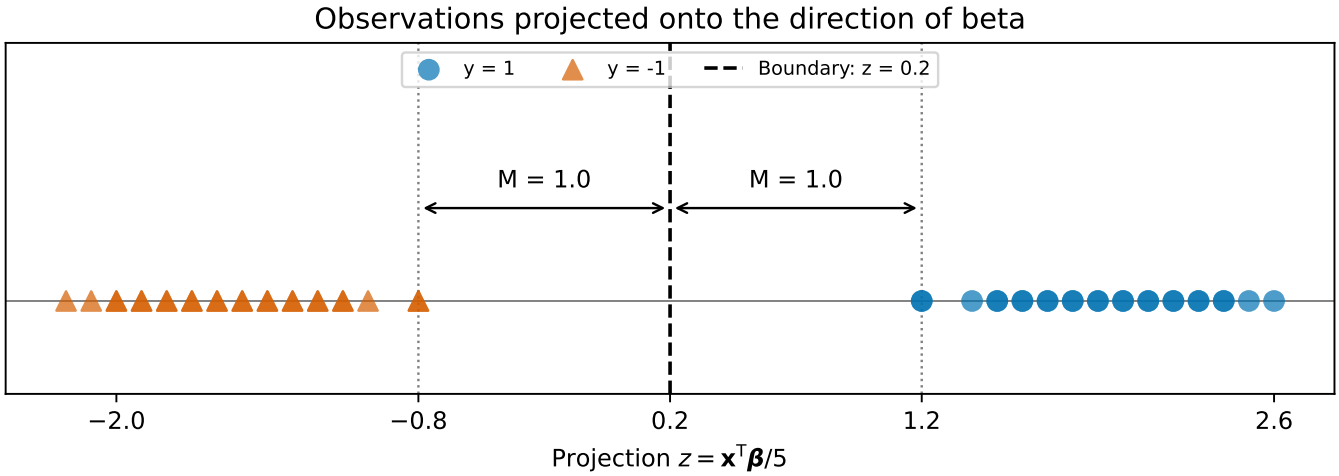
(a) Since $\|\beta\| = 5$, I project each observation onto the unit direction $\beta/5$. Its position is $z = \mathbf{x}^\mathsf{T}\beta/5$, so $f(\mathbf{x}) = -1 + 5z$ and the boundary is at $z = 0.2$.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

data_4d = pd.read_csv("margin-geometry-4d.csv")
predictors = data_4d[["x1", "x2", "x3", "x4"]].to_numpy()
beta = np.array([1., 2., -2., 4.])
beta0 = -1.
beta_norm = np.linalg.norm(beta)
projection = predictors @ (beta / beta_norm)
boundary = -beta0 / beta_norm
distances = np.abs(projection - boundary)
margin = distances.min()
closest = np.isclose(distances, margin)

fig, ax = plt.subplots(figsize=(8, 3))
for label, color, marker in [(1, "#0072B2", "o"), (-1, "#D55E00", "^")]:
    selected = data_4d["y"].to_numpy() == label
    ax.scatter(projection[selected], np.zeros(selected.sum()),
               color=color, marker=marker, s=65, alpha=0.7,
               label=f"y = {label}", zorder=3)
ax.axhline(0, color="gray", linewidth=0.8)
ax.axvline(boundary, color="black", linestyle="--", label="Boundary: z = 0.2")
for position in [boundary - margin, boundary + margin]:
    ax.axvline(position, color="gray", linestyle=":", linewidth=1)
    ax.annotate("", xy=(boundary, 0.18), xytext=(position, 0.18),
                arrowprops={"arrowstyle": "<->", "color": "black"})
    ax.text((boundary + position) / 2, 0.22, f"M = {margin:.1f}", ha="center")
ax.set(ylim=(-0.18, 0.5), yticks=[],
       xticks=[-2, -0.8, 0.2, 1.2, 2.6],
       xlabel=r"Projection $z = \mathbf{x}^{\mathsf{T}}\boldsymbol{\beta}/5$",
       title="Observations projected onto the direction of beta")
ax.legend(loc="upper center", ncol=3, fontsize=8)
plt.tight_layout()
plt.show()

print(f"Geometric margin M = {margin:.2f}")
print("Closest observations:", data_4d.loc[closest, "observation"].tolist())
```




```
Geometric margin M = 1.00
Closest observations: [4, 15, 33, 36]
```

The nearest projected positions are -0.8 and 1.2, each 1 unit from the boundary at 0.2. Thus, $M = 1$. Observations **4, 15, 33, and 36** attain this distance. Points with the same projection overlap in the plot.

(b) The smallest functional margin is 5. Dividing both coefficients and intercept by 5 preserves the boundary and makes the smallest functional margin equal to 1:

$$\beta_{\text{SVM}} = (0.2,\ 0.4,\ -0.4,\ 0.8)^\top, \qquad \beta_{0,\text{SVM}} = -0.2.$$

```
scores_4d = beta0 + predictors @ beta
minimum_functional_margin = (data_4d["y"].to_numpy() * scores_4d).min()
beta_svm = beta / minimum_functional_margin
beta0_svm = beta0 / minimum_functional_margin
normalized_margins = data_4d["y"].to_numpy() * (beta0_svm + predictors @ beta_svm)
print(f"Smallest original functional margin: {minimum_functional_margin:.2f}")
print("Normalized beta:", beta_svm)
print(f"Normalized intercept: {beta0_svm:.2f}")
print(f"Smallest normalized functional margin: {normalized_margins.min():.2f}")
```

```
Smallest original functional margin: 5.00
Normalized beta: [ 0.2  0.4 -0.4  0.8]
Normalized intercept: -0.20
Smallest normalized functional margin: 1.00
```

Question 3

(a) The assigned skill test used seed 432, a stratified 30% test set, and five shared stratified cross-validation folds. It selected PCA(4) + QDA, with a CV error of 25.98% and a test error of 25.90%. The run completed without an early stop or steer message.

(b) Overall assessment

Overall, the student's LDA/QDA skill is well organized and provides clear instructions for model comparison, cross-validation, and reporting. I especially liked that it uses the same folds for every candidate, performs preprocessing within training folds, and reserves a separate test set. The student's report selected PCA(5) + QDA with a CV error of 24.7% and a test error of 28.8%, while the assigned skill test selected PCA(4) + QDA with a CV error of 25.98% and a test error of 25.90%. These differences do not necessarily indicate an error: the runs used different seeds and software, and the student's report used the first version of the skill while the assigned skill test used the final version. However, the comparison highlights areas where the skill could be improved.

1. **Assess model selection stability.** The student's report selected five principal components, while the assigned skill test selected four. Although the different settings may explain this, it raises questions about how stable the model selection is. I recommend adding repeated stratified cross-validation within the training data to evaluate how often each model is selected and how stable the performance rankings are across repetitions. The test set should remain untouched during this comparison.

2. **Evaluate sensitivity and specificity tradeoffs:** The student's original report achieved 64.6% sensitivity, while the assigned skill test achieved 50.0%, missing 24 of the 48 test cases with CHD. The skill already reports sensitivity and specificity and acknowledges the limitations of the 0.5 threshold, but it does not investigate alternative thresholds. I recommend evaluating threshold tradeoffs using training cross-validation, while keeping misclassification rate as the primary selection criterion and leaving the test set untouched until final evaluation. This would show whether increased sensitivity comes at an acceptable loss of specificity.

3. **Make assumption checks more actionable.** The skill instructs the agent to identify skewed or discrete predictors, but it does not specify how to follow up on these findings. Both reports noted skewness in variables such as tobacco and alcohol. Since LDA and QDA assume normality within each class, pooled skewness alone does not establish a violation. I recommend adding within-class distribution checks and evaluating appropriate transformations through cross-validation, with any estimated transformation parameters fitted within each training fold. This would make the checks more informative without assuming transformations will improve performance or moving outside the permitted LDA/QDA methods.

(c) GitHub review issue: Homework 7 LDA/QDA skill peer review.